

Cube Trainer: Midterm Report

1. Repository Overview

<https://github.com/ntsai04/cubetrainer/tree/main>

CUBETRAINER

run.py

The run.py file runs the main loop that updates the user's UI and makes calls to the detection and classification components of the vision implementation.

vision

detection

shape_detector.py

The shape_detector class locates a Rubik's cube face and returns its 4 corner coordinates. The detect_face method preprocesses the frame, uses Canny edge detection, finds contours, and uses helper methods to keep the largest, most square-like contour, ultimately the face.

colors

classifier.py

The classifier class extracts and classifies the colors on the 9 squares of the passed face. The sample_face_colors method uses the face's 4 corners to warp a 3x3 grid of 900 pixel resolution onto the input frame. It then samples each grid cell and uses simple HSV color comparison to classify the 3x3 face.

backend

The backend section of our project will be completed in the second half of the semester. It will use the color arrays produced by the vision components to recognize patterns and output the necessary algorithm needed.

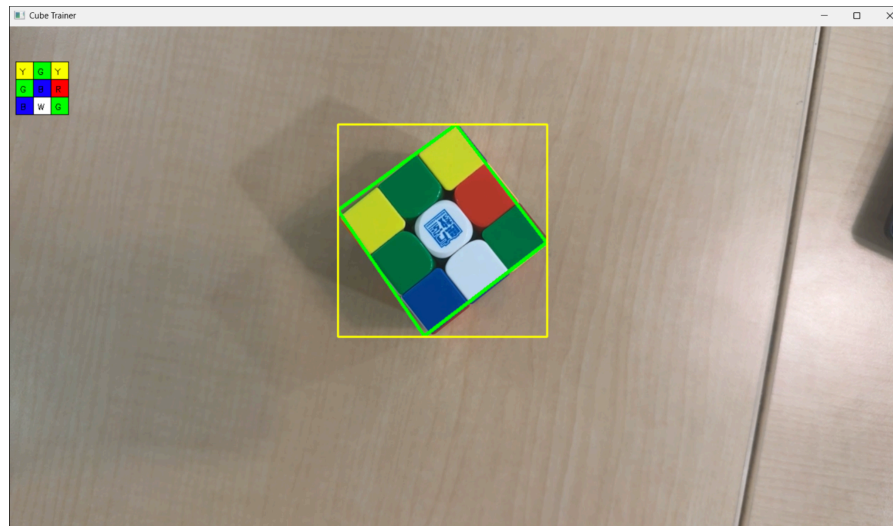
** So far our project utilizes Python packages OpenCV (cv2) and NumPy.

2. Current Progress & Limitations

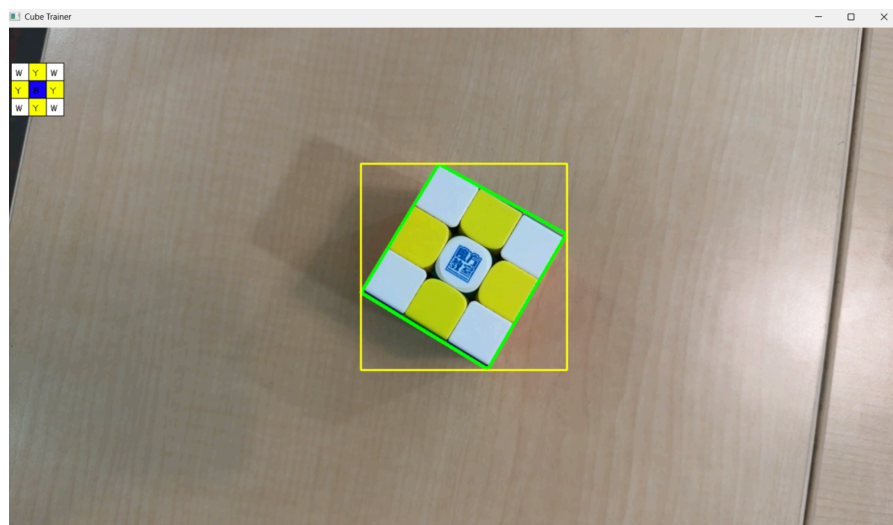
Included below are images from a test run that illustrate the current state of the project and some of the technical limitations we have encountered.

Nathan Tsai
Yibing Shen

- Able to detect edges and classify colors with in-plane rotations and fairly clean background.

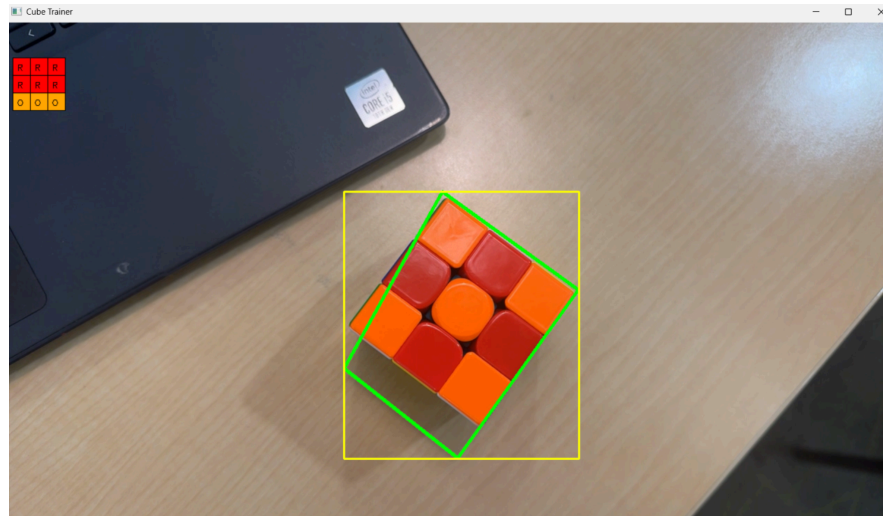


- Unable to recognize logo and mis-classified as other colors

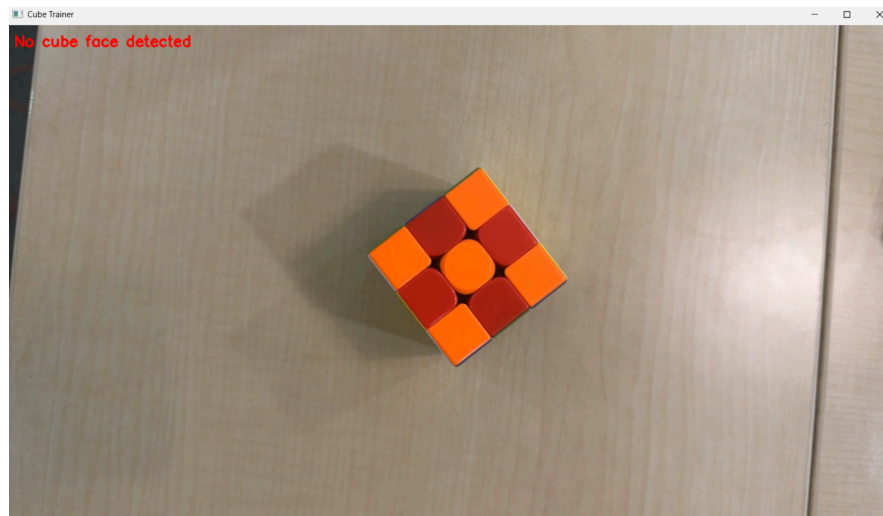


Nathan Tsai
Yibing Shen

- Color-classification is less accurate with colors with similar hsv values, most noticeably Red & Orange.

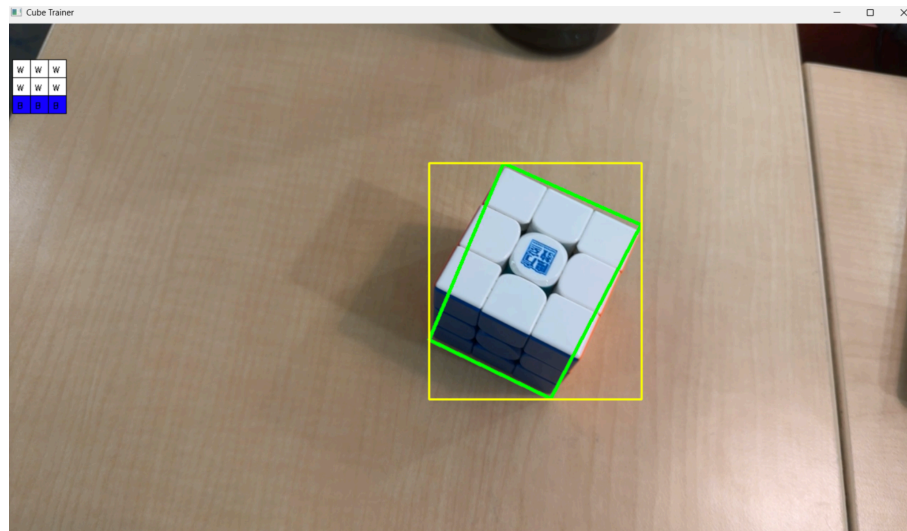


- Unable to stably recognize edges with Red-Orange patterns.

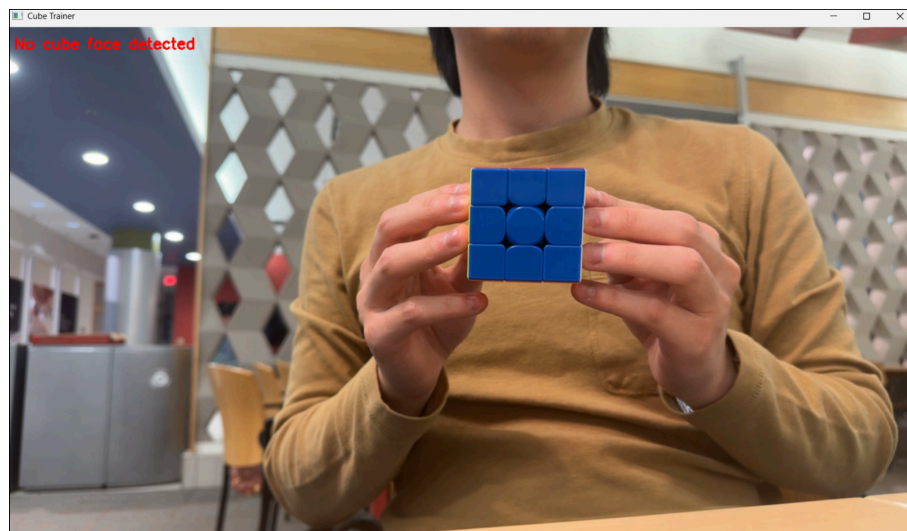


Nathan Tsai
Yibing Shen

- Unable to detect edges correctly with out of plane rotation.



- Unable to recognize cube with complex background.



3. Changes to our Proposal Scope and Next Steps

- **Refinement of Goals:**

Aiming for dynamic sampling and tracking of multiple faces and turns proved complex and quite ambitious. However, we would like to significantly improve the quality of face scanning on the user end. Most other implementations force a specific scanning order and most notably orientation; we will simply ask for each face (specified by center color) and orient the cube in the backend.

- **Improved Edge Detection and Color Sampling:**

We will continue to experiment with the parameter space to use values that most accurately and *efficiently* find the edges and colors. This will make the user experience seamless and reduce the need for rescanning. To accomplish this we may look into ML-based color classification to ensure accuracy in this important step (KMeans clustering or small CNN).

- **Backend Development:**

Initial plans include using the color arrays produced by the vision components to create a digital model of the cube automatically and use a database to pattern search and return algorithms.